

Week 3. Problem set (solutions by Karim Khabibrakhmanov)

1. Consider a hash table with 13 slots and the hash function $h(k) = (k^2 - k + 11) \bmod 13$. Show the state of the hash table after inserting the keys (in this order)

6, 28, 19, 15, 20, 33, 12, 17, 10, 13, 3, 39

with collisions resolved by *linear probing* [CLRS, §11.4].

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Key	28	15	6	19	20	33	12	3	39		17	10	13

Answer:

<i>Index</i>	<i>0</i>	<i>1</i>	<i>2</i>	<i>3</i>	<i>4</i>	<i>5</i>	<i>6</i>	<i>7</i>	<i>8</i>	<i>9</i>	<i>10</i>	<i>11</i>	<i>12</i>
<i>Key</i>	<i>28</i>	<i>15</i>	<i>6</i>	<i>19</i>	<i>20</i>	<i>33</i>	<i>12</i>	<i>3</i>	<i>39</i>		<i>17</i>	<i>10</i>	<i>13</i>

Justification (optional):

For the following keys (k), we will use the hash function $h(k) = (k^2 - k + 11) \bmod 13$. Since we are employing linear probing, if the calculated index is already occupied, we will insert the key at the next available empty index:

- 1) $h(6) = 2$ – Insert at index 2.
- 2) $h(28) = 0$ – Insert at index 0.
- 3) $h(19) = 2$ – Insert at index 3.
- 4) $h(15) = 0$ – Insert at index 1.
- 5) $h(20) = 1$ – Insert at index 4.
- 6) $h(33) = 1$ – Insert at index 5.
- 7) $h(12) = 0$ – Insert at index 6.
- 8) $h(17) = 10$ – Insert at index 10.
- 9) $h(10) = 10$ – Insert at index 11.
- 10) $h(13) = 11$ – Insert at index 12.
- 11) $h(3) = 4$ – Insert at index 7.
- 12) $h(39) = 11$ – Insert at index 8.

2. Consider a dictionary that maps an integer (e.g. group number) and a short¹ string (e.g. student email) to a number (e.g. grade), and is implemented as a hashtable T of hashtables:

- for an integer k , if T contains k , then $T[k]$ is a hashmap with string keys
- for a string s , if $T[k]$ contains s , then $T[k][s]$ is a floating point number

Compute average case time complexities of successful and unsuccessful search in this hashtable of hashtables with the different possible combinations of chaining and open addressing, assuming

- (a) *independent uniform hashing* for hashtables with separate chaining [CLRS, §11.2]
- (b) *independent uniform permutation hashing* for hashtables with open addressing [CLRS, §11.4]
- (c) for the hashtable T , load factor α and size n
- (d) for the hashtables $T[k]$, average load factor β and average size m

	Successful search	Unsuccessful search
T uses separate chaining and each $T[k]$ uses separate chaining	$O(1 + \alpha + \beta)$	$O(1 + \alpha + \beta)$
T uses separate chaining and each $T[k]$ uses open addressing	$O(1 + \alpha + \frac{1}{\beta} \ln(\frac{1}{1-\beta}))$	$O(1 + \alpha + \frac{1}{1-\beta})$
T uses open addressing and each $T[k]$ uses separate chaining	$O(\frac{1}{\alpha} \ln(\frac{1}{1-\alpha}) + 1 + \beta)$	$O(\frac{1}{1-\alpha} + 1 + \beta)$
T uses open addressing and each $T[k]$ uses open addressing	$O(\frac{1}{\alpha} \ln(\frac{1}{1-\alpha}) + \frac{1}{\beta} \ln(\frac{1}{1-\beta}))$	$O(\frac{1}{1-\alpha} + \frac{1}{1-\beta})$

¹Assume that strings have a relatively small constant maximum size.

3. (+0.5% extra credit) Compute the average case time complexity of the following algorithm:

```
1 secret(M, k):
2   Q := new linked queue
3   while M.containsKey(k) and k > 0
4     item := M.get(k)
5     k := min(item.value, k) - 1
6     if k == item.value - 1
7       Q.offer(item)
8   while not Q.isEmpty()
9     M.remove(Q.poll())
10  return k
```

M is a hashtable of size n with load factor α that resolves collisions by *separate chaining* [CLRS, §11.2]. The answer **must** use O -notation and may depend on n , k , and α . Assume *independent uniform hashing* and worst case for the contents (values) of the input map M .

Justify your answer (3–5 sentences). Detailed proof is not required.

Answer: $O((\alpha + 1) \cdot (k + 1))$

Justification:

In the worst case, the first while loop runs $k + 1$ times, and the total complexity of the this loop is $O(a + 1)$. The second while loop also run $k + 1$ times, and its total complexity of the this loop is $O(a + 1)$. Therefore, the overall complexity of following algorithm will be:

$$O((\alpha + 1) \cdot (k + 1)) + O((\alpha + 1) \cdot (k + 1)) = O((\alpha + 1) \cdot (k + 1))$$

References

- [CLRS] Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C., 2022. *Introduction to algorithms, Fourth Edition*. MIT press.